

How I Built a Melee Combo System in Lyra

Understanding and Adapting Unreal’s Lyra Framework for Combat animation

Getting Started

Carlo Andrei Mercado Morales

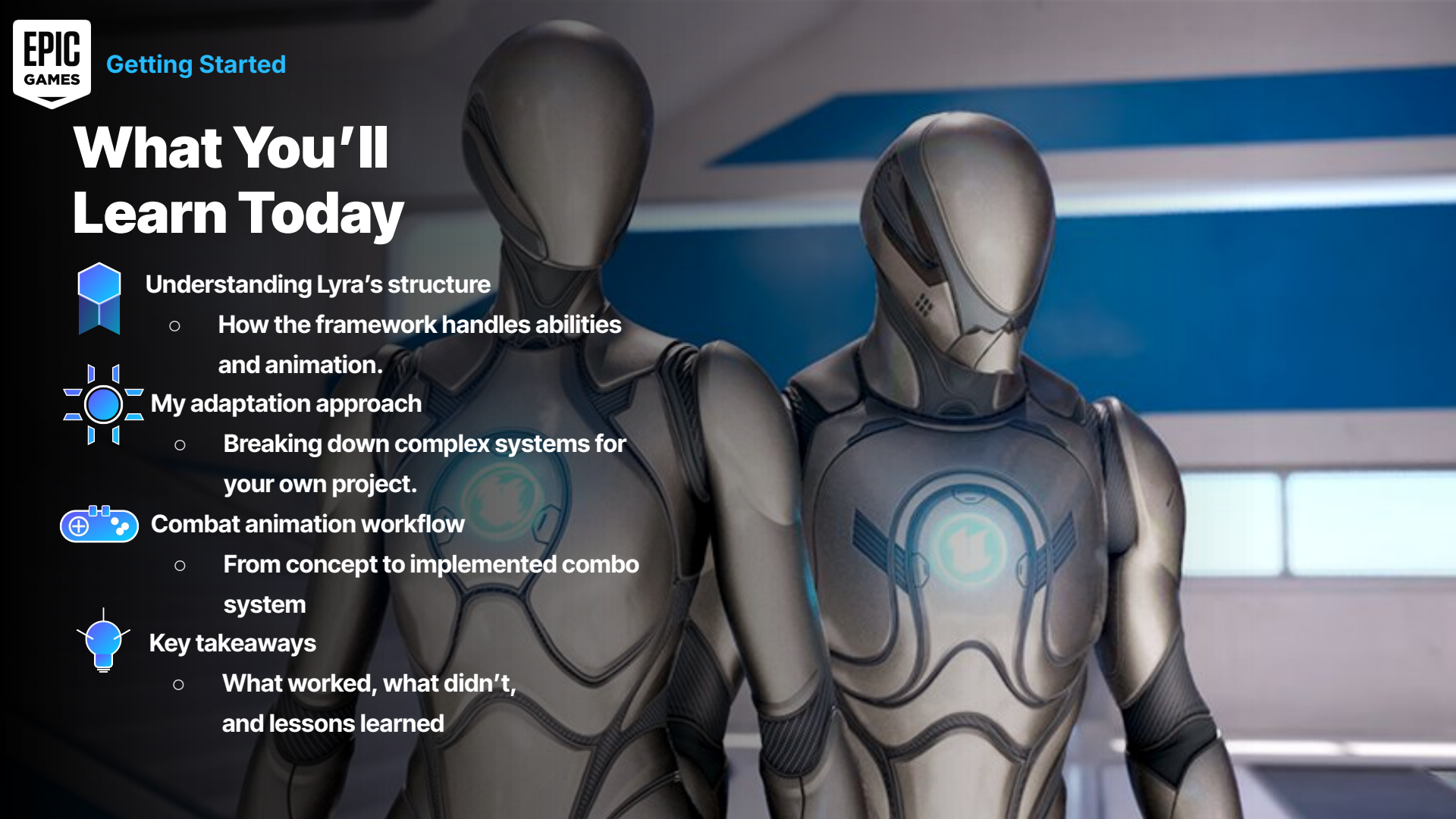
About Me

- Lead Gameplay Animator at Ubisoft Bordeaux
- 12+ years in game animation across multiple studios
- Lead Animator on Assassin's Creed Shadows & Claws of Awaji DLC
- Epic Games Instructor (Unreal Animation Fellowship 2023, Game Fellowship EMA 2025)
- Specializing in UE5 Animation Blueprints and gameplay systems



What You'll Learn Today

- Understanding Lyra's structure
  - How the framework handles abilities and animation.
- My adaptation approach
  - Breaking down complex systems for your own project.
- Combat animation workflow
  - From concept to implemented combo system
- Key takeaways
  - What worked, what didn't, and lessons learned



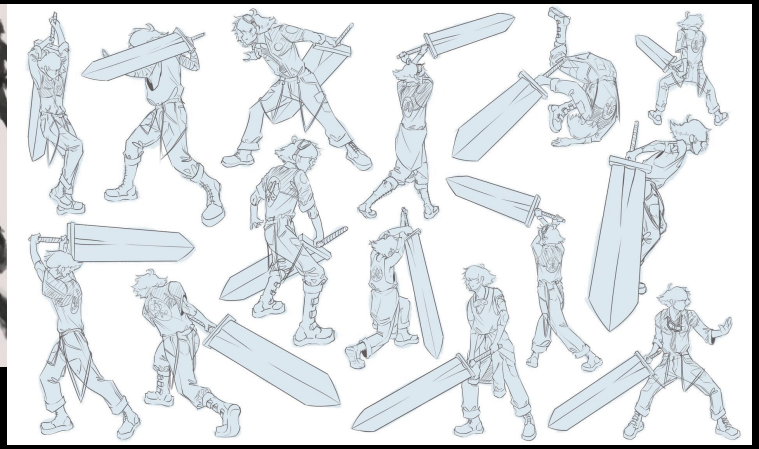
The End Result

- Full 3 combo system with earlier And late cancel of actions.
- Full Jog Locomotion.
- Jump attack.
- Rush attack.
- Dodge system.

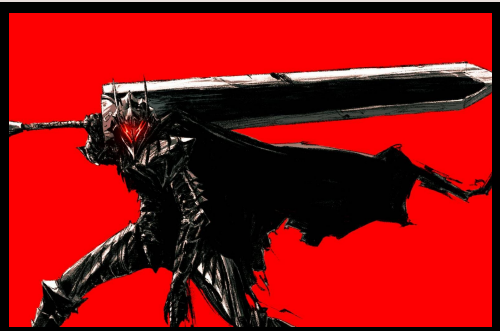


The references





- Monster Hunter World | Great Sword Tutorial
- Elden Ring: Greatsword (Weapon Showcase Ep.6)
- Nosferatu Zodd | Berserk: The Golden Age Arc - Memorial Edition







Understanding Lyra



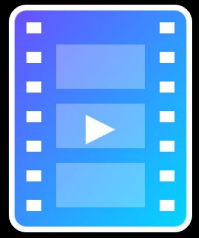
What is Lyra?

- Epic's production-ready sample project
- Built with AAA best practices
- Modular gameplay systems using GAS (Gameplay Ability System)
- Full multiplayer shooter framework

Why Lyra?

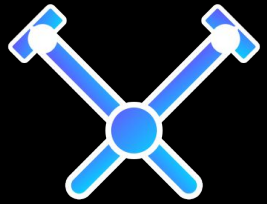
- Professional starting point - skip months of setup
- Real production code - learn from Epic's engineers
- Modular design - easy to adapt for your game
- Active development - continuously updated by Epic

What We'll Cover in This Demo



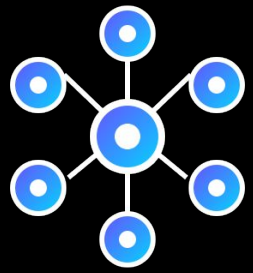
Animation System

- Animation Blueprint & linked layer setup in Lyra
- Montage setups for Rolls
- Connecting the Linked layer to the Lyra system



Combat Architecture

- Melee attack anatomy overview
- Melee combo architecture & flow
- Data structure of weapons
- Animation notifies for custom melee system



Gameplay Ability System (GAS)

- Gameplay ability setups & effects
- Custom abilities: my combo system approach
- Gameplay tags (Lyra's & custom tags)

Animation System Architecture

Base Animation Blueprint

- ABP_Mannequin_Base (Untouched)
- Core locomotion logic



Path: /Game/Characters/Heroes/Mannequin/Animations

Custom Linked AnimLayer

- ABP_GiantSword-AnimLayers (Created for weapon)
- Connected to base via Linked Layer system



Path: /Game/Characters/Heroes/Mannequin/Animations/Locomotion/GiantSword

Demo Scope Note

- Focus: JOG locomotion only
- Omitted: Walk animations (4 directions, 4 pivots, 4 starts, 4 stops)

Ground Locomotion Animations



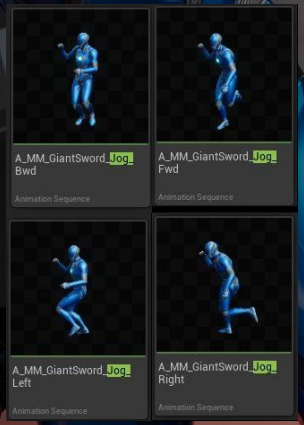
IDLE

- 1 animation
- Idle_Ready
- Neutral pose
- Root motion: OFF



Turn in Place

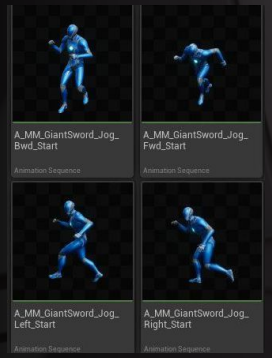
- TurnL-90, TurnR-90
- 90° rotations
- Root Motion: ON



Jog Cycles

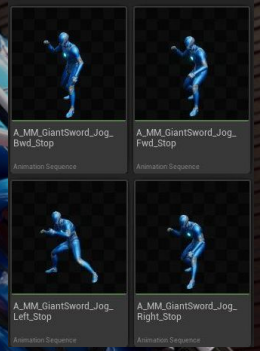
- 4 animations
- Jog_Fwd/Bwd
- Jog_Left/Right
- Looping
- Root motion: ON

Ground Locomotion Animations



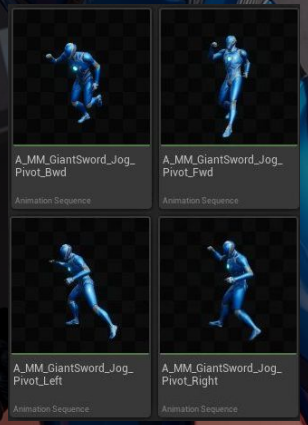
Jog Starts

- 4 animations
- Jog_Fwd/Bwd
- Jog_Left/Right
- Idle→passing pose
- Root motion: ON



Jog Stop

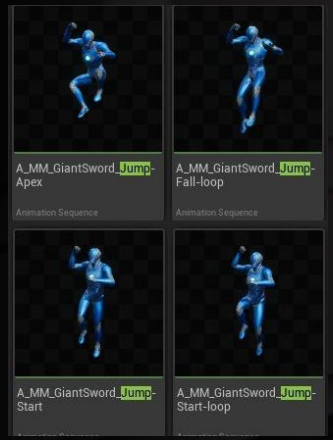
- 4 animations
- Jog_Fwd/Bwd
- Jog_Left/Right
- passing pose→Idle
- Root motion: ON



Jog Pivots

- 4 animations
- Jog_Fwd/Bwd
- Jog_Left/Right
- Cycle→ invert direction
- Root motion: ON

Jump Animations



Jump States

- 4 animations
- 2 Loops
  - Fall, Jump
  - Root Motion:OFF
- 2 transitions.
  - Jump-Start, Jump Apex
  - Root motion: ON



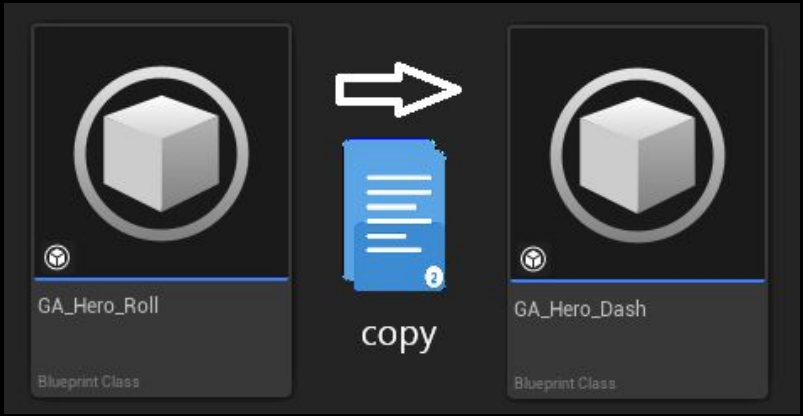
Jump Recovery

- 1 animations
- Additive
- Transition to ground
- Root motion: ON

Blueprint Setup:Rolls

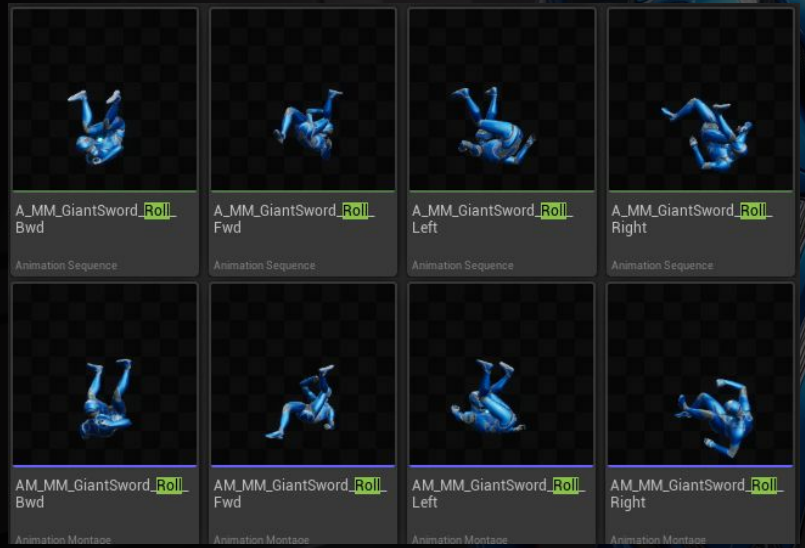
- For the Hero Roll gameplay Ability I duplicate the **GA_Hero_Dash**, call **GA_Hero_Roll**, main reason I want to modify in the future the logic to have immunity frames but without modifying the original blueprint.

- This GA plays instead of the GA_Hero_Dash, and use the same logic in the moment with different values.



Animation System Architecture

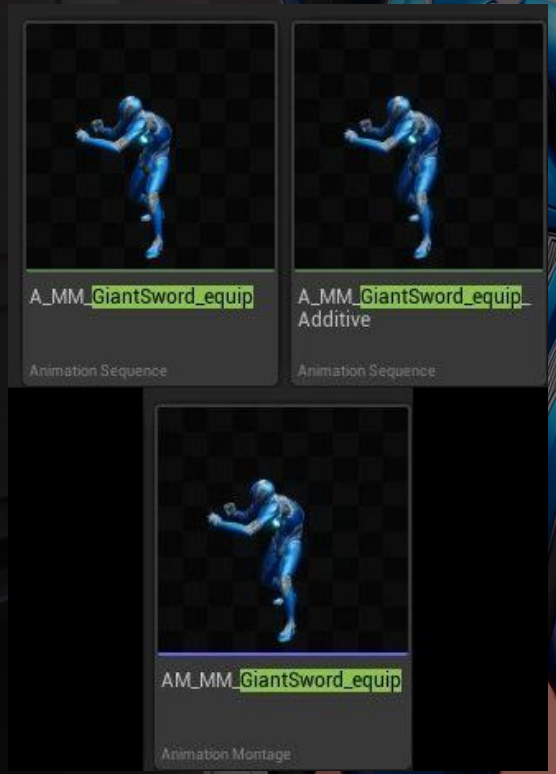
Roll Animations



Rolls

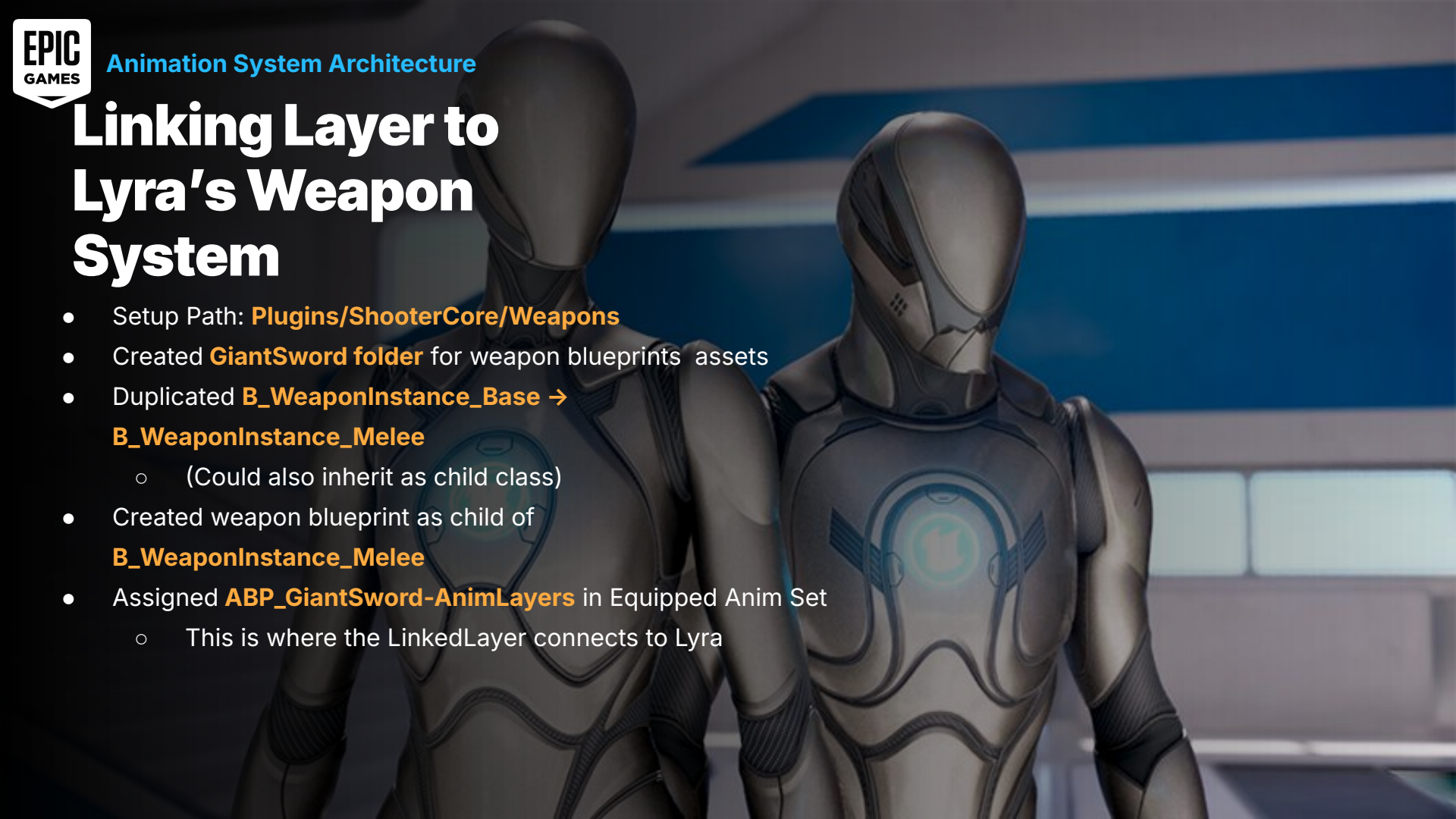
- 4 animations
- 1 montage per animation
- Idle→Roll→Idle
- Default Slot
- Root motion: OFF

Ground Locomotion Animations



Equip

- 1 Additive Animation 1 Non Additive
- Inplace
- Root motion: OFF
- Idle Unarm→equip action→IdleGS
- NonAdditive plays in the UpperBody Slot
- Additive Plays in the UpperBodyAdditive Slot.



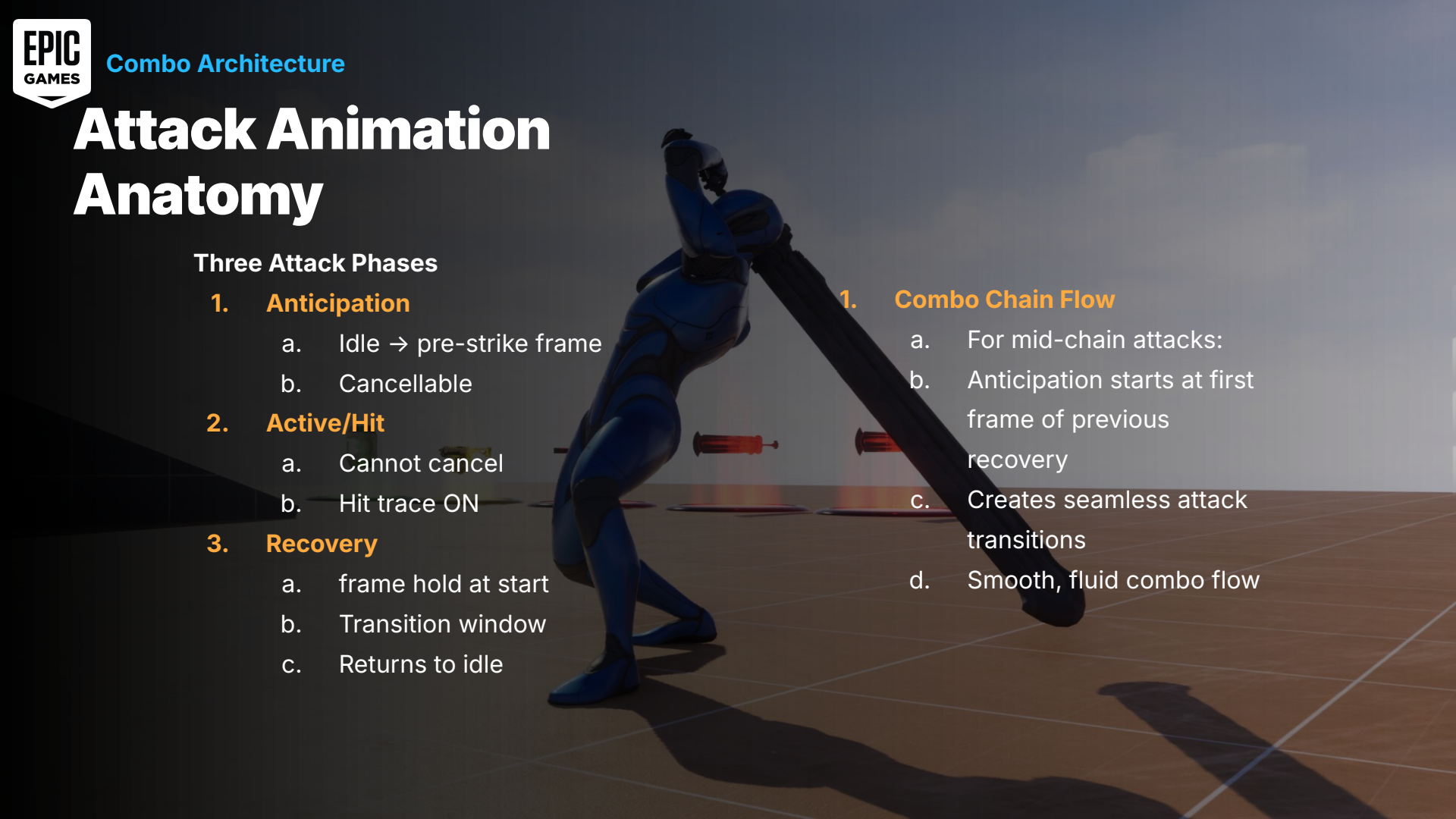
Animation System Architecture

Linking Layer to Lyra's Weapon System

- Setup Path: **Plugins/ShooterCore/Weapons**
- Created **GiantSword folder** for weapon blueprints assets
- Duplicated **B_WeaponInstance_Base →** **B_WeaponInstance_Melee**
    - (Could also inherit as child class)
- Created weapon blueprint as child of **B_WeaponInstance_Melee**
- Assigned **ABP_GiantSword-AnimLayers** in Equipped Anim Set
    - This is where the LinkedLayer connects to Lyra

Combo Architecture

Attack Animation Anatomy



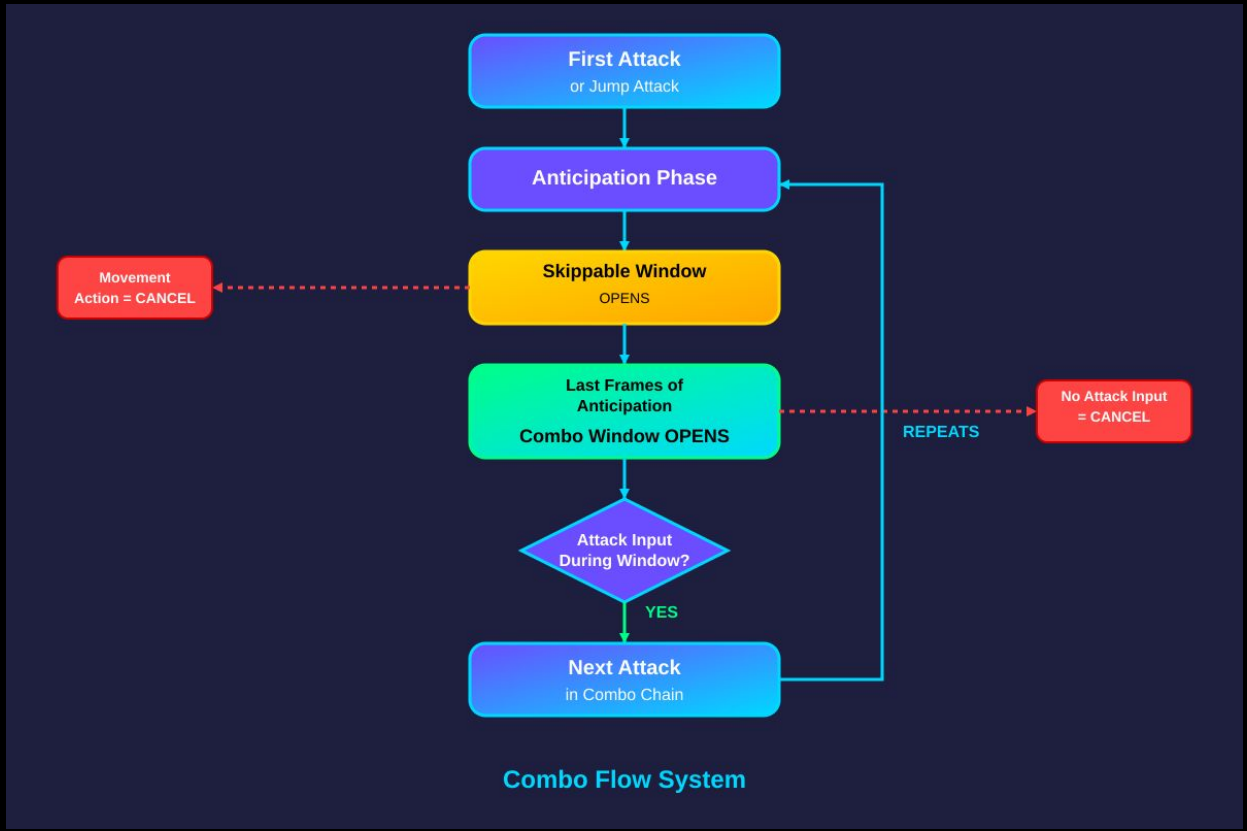
Three Attack Phases

1. **Anticipation**
   a. Idle → pre-strike frame
   b. Cancellable
2. **Active/Hit**
   a. Cannot cancel
   b. Hit trace ON
3. **Recovery**
   a. frame hold at start
   b. Transition window
   c. Returns to idle

1. **Combo Chain Flow**
   a. For mid-chain attacks:
   b. Anticipation starts at first frame of previous recovery
   c. Creates seamless attack transitions
   d. Smooth, fluid combo flow

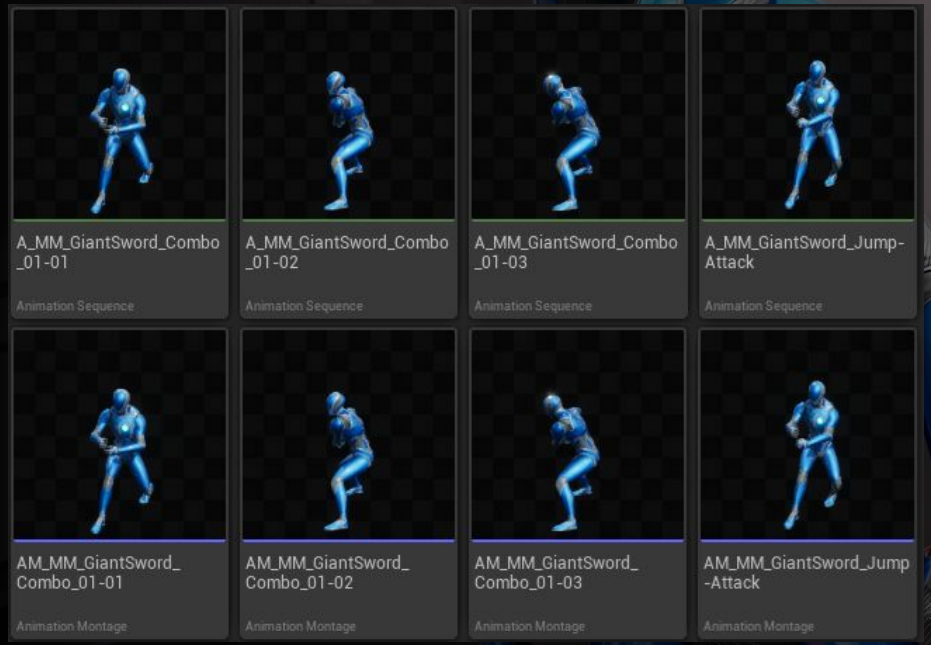
Combo Architecture

Combo Flow



Combo Architecture

Combo Animations



Combo Animations

- 4 animations
  - 3 ground attacks 1 Jump attack
- 1 montage per animation
- Anticipation→Active or Attack → recovery
- Default Slot
- Motion Warping
- Root motion: ON

Combo Architecture

Blueprint Setup:Combo

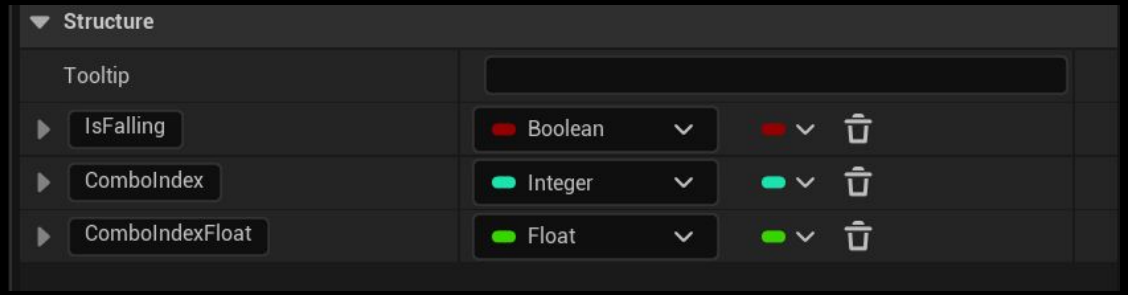
- I create a **GA_MeleeWeapon_DamageBase**
- I did a logic for the weapon trace in this new blueprint and create a child name **GA_Melee-Combo** this will be the parent class of any weapon Ability that needs to use combo.
- I make a child for **GA_Melee-Combo_GlantSword**
- This serves as the foundation for implementing:
  - Combo sequence logic
  - Attack tracking
  - Event-driven animation control
  - Hit detection system

Combo Architecture

Blueprint Setup:Combo

- As part of the combo logic I create a **Chooser table**, that help me to organize my combo sequence in a data driven structure.
- And I create a **Structure** that contains the input data I need to pass to the Chooser table.



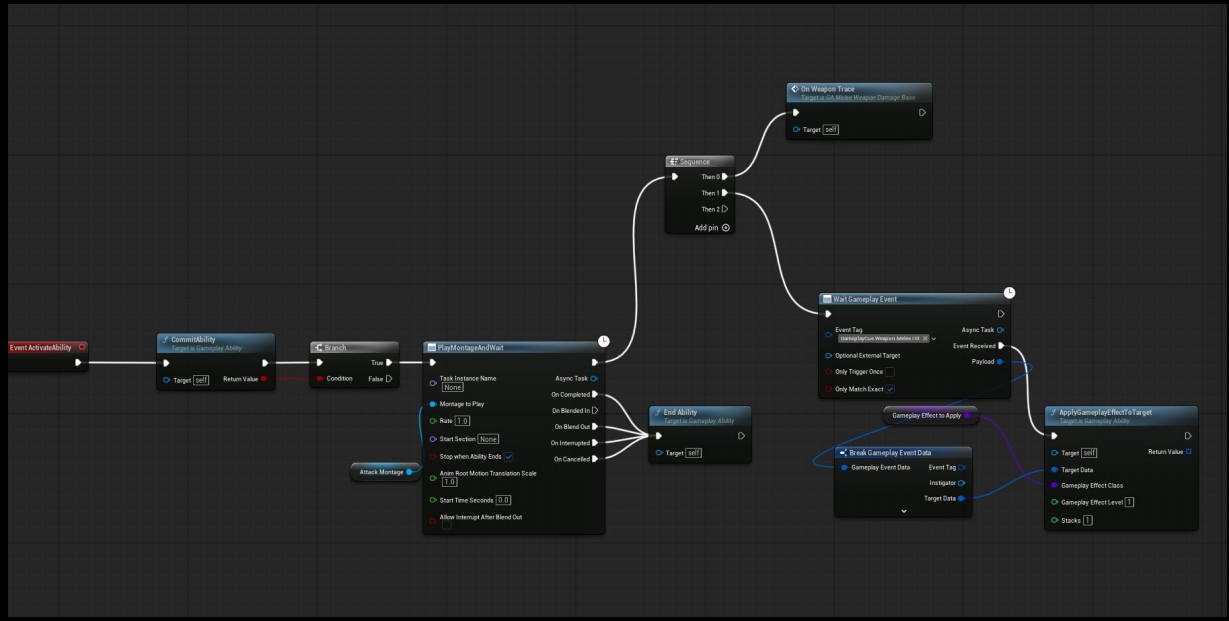


Combo Architecture

Blueprint Setup:ChargeAttack

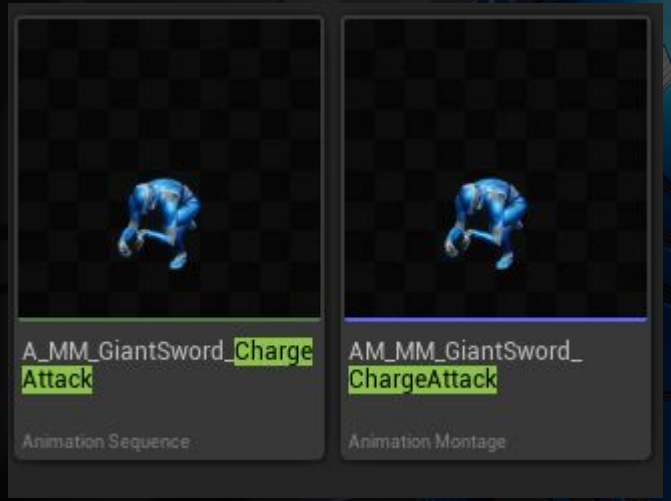
- I create a Child of the **GA_MeleeWeapon_DamageBase** call **GA_ChargeAttack_GiantSword**

- This is a basic Gameplay ability focus on play a Charge attack montage animation
- And reuse the logic of the parent for the Melee trace



Combo Architecture

Charge Attack Animations



Charge Attack

- 1 animation
- 1 montagen
- Anticipation→Charge→Active or Attack → recovery
- Default Slot
- Motion Warping
- Root motion: ON

Combo Architecture

Notifies

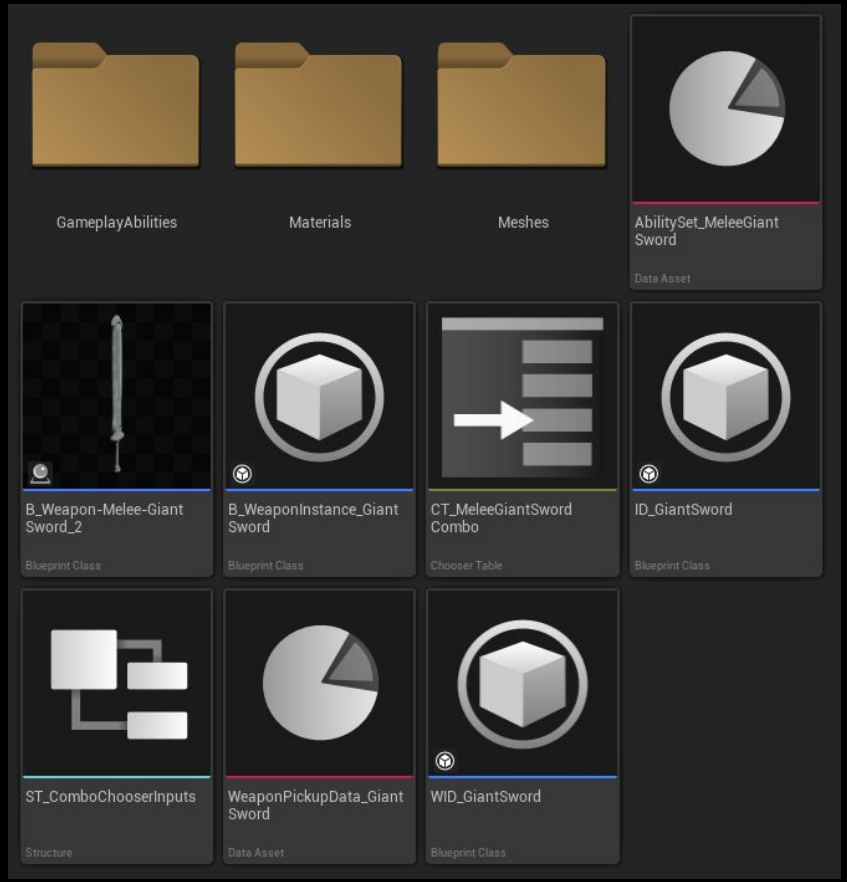
- 2 Notifies created.
- **AN_SkippableWindow** send
  - Event.Montage.Timing.Skippable
  - Event.Montage.Timing.UnSkippable
- **AN_ComboWindow** send
  - Event.Montage.Combo.Open
  - Event.Montage.Combo.Close
- Extra notify State by the Plugging DidIHit.
  - **Melee Trace**, is in charge to trace the hit during the animation



Folder Structure

Final Folder Structure

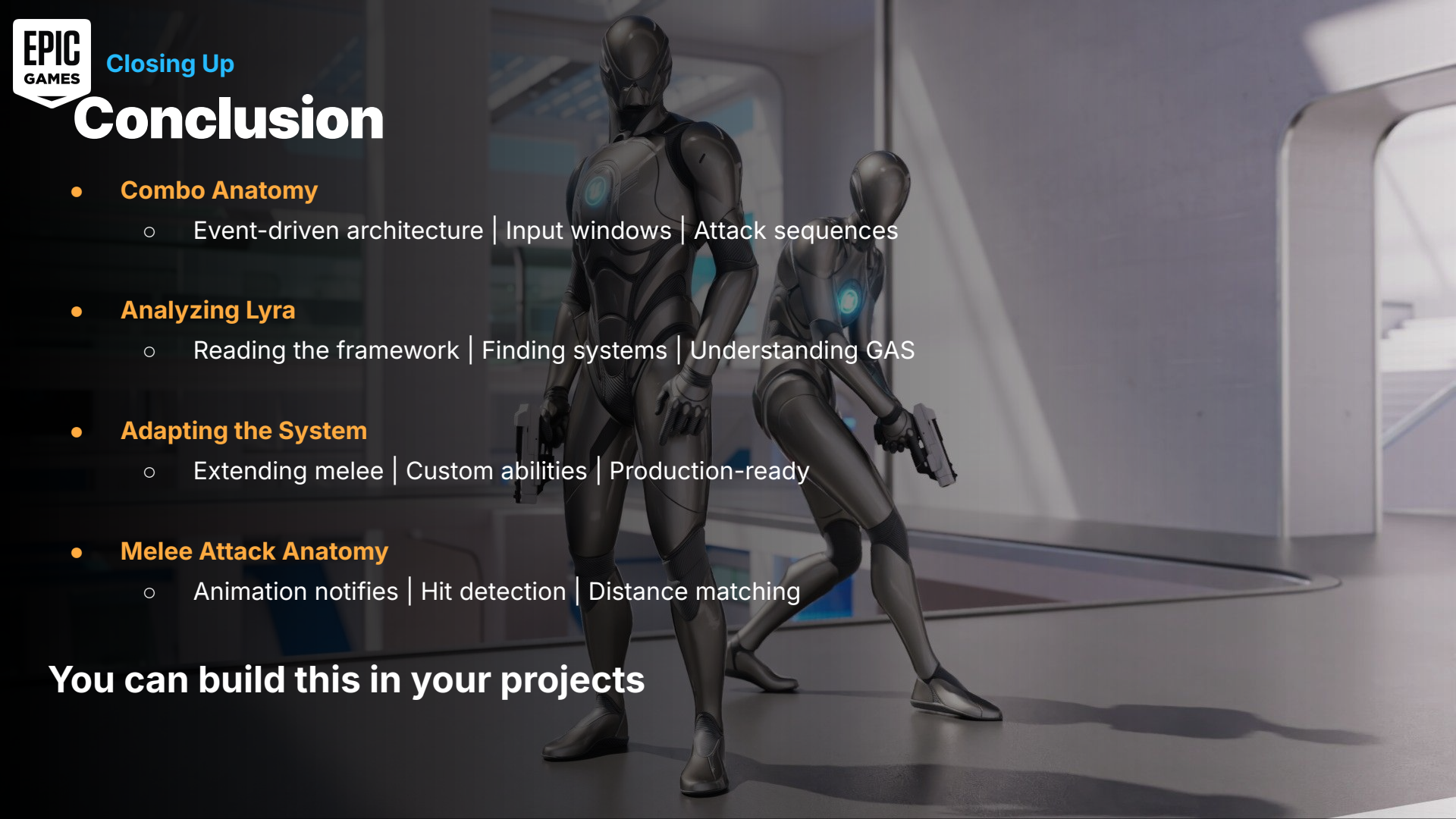
- **Folders:** GameplayAbilities | Materials | Meshes
- **Key Files:**
  - AbilitySet → Weapon abilities
  - B_Weapon-Melee-GiantSword_2→ Player spawn
  - B_WeaponInstance_GiantSword→ Pickup
  - CT_MeleeGiantSwordCombo→ Combo table
  - ST_ComboChooserInputs→ Structure Input values CT
  - WeaponPickupData_GiantSword→pickup data of the weapon
  - ID_GiantSword → Inventory
  - WID_GiantSword → Weapon ID



Closing Up

Conclusion

- **Combo Anatomy**
  - Event-driven architecture | Input windows | Attack sequences
- **Analyzing Lyra**
  - Reading the framework | Finding systems | Understanding GAS
- **Adapting the System**
  - Extending melee | Custom abilities | Production-ready
- **Melee Attack Anatomy**
  - Animation notifies | Hit detection | Distance matching



You can build this in your projects